

B3 Series B

Running cellranger count and reading the web_summary

Finishing count is the easy part — knowing whether the data is usable is the skill.

About 40 min | Prerequisites: B2 | Data: PBMC 1k FASTQs

Two web_summaries: one healthy, one disaster

Sample A • web_summary.html

Estimated Number of Cells	1,187
Mean Reads per Cell	54,102
Median Genes per Cell	2,412
Valid Barcodes	97.6%
Confidently Mapped to Transcriptome	62.3%

alerts: 0

Sample B • web_summary.html

Estimated Number of Cells	9,315
Mean Reads per Cell	6,890
Median Genes per Cell	342
Valid Barcodes	71.2%
Confidently Mapped to Transcriptome	28.4%

alerts: 4

Can you tell in 30 seconds which one is unusable — and why?

Same assay, same pipeline — the whole difference is written on this one page

WHERE THIS EPISODE STARTS

**web_summary is the health report of your run.
If you can't read it, downstream polish is wasted.**

If the matrix is broken, Seurat will not notice — it will faithfully analyse garbage.

What you will learn

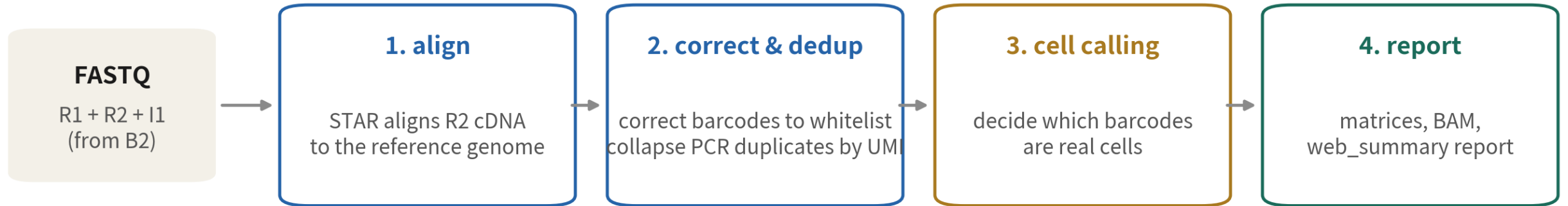
- 1 Understand every flag of cellranger count and run it on PBMC 1k
- 2 Read the web_summary zone by zone: ten key metrics, knee-plot shapes, alerts
- 3 Know what every file in outs/ is for — especially filtered vs raw

01

What count actually does

Before typing, know the four jobs behind one Enter

One command, four jobs

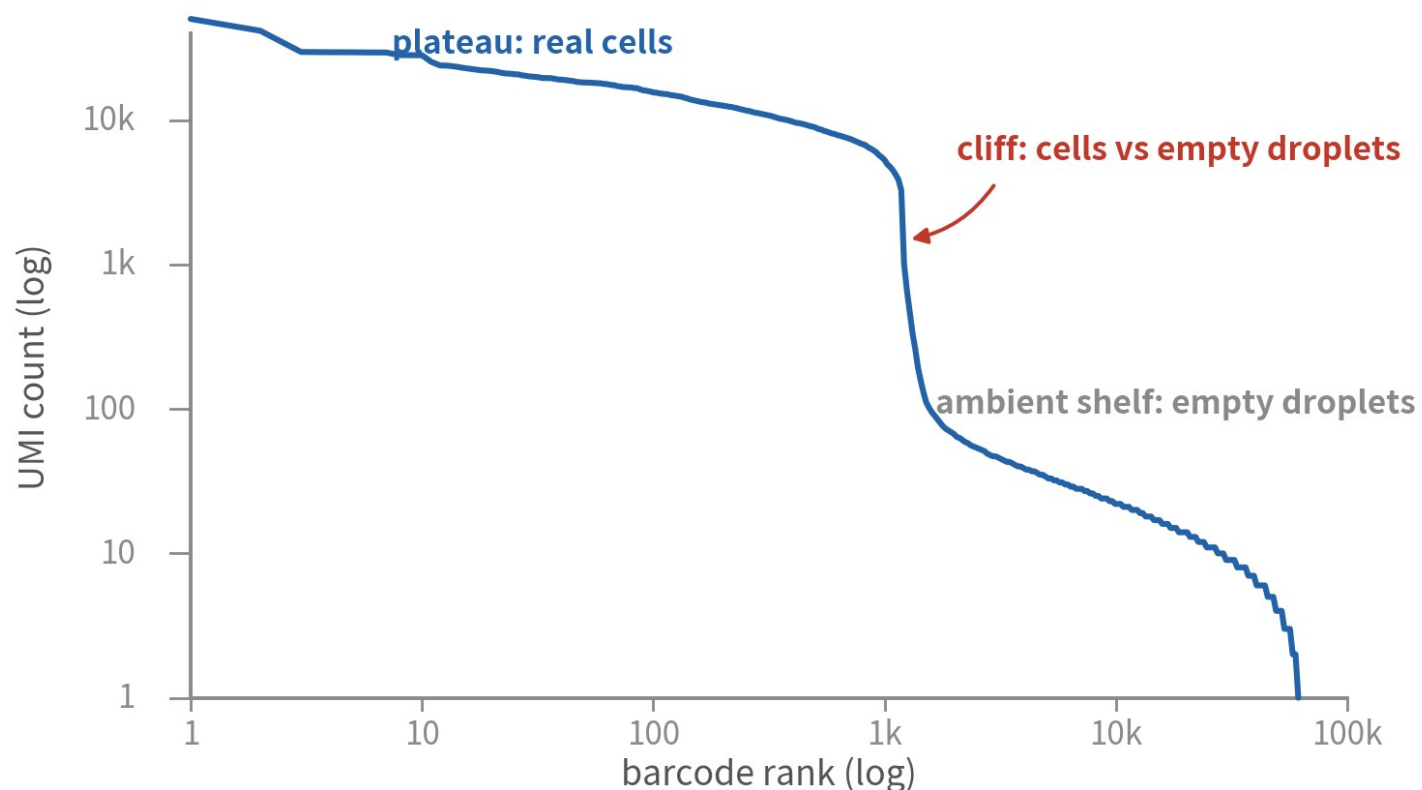


One command runs all four — and web_summary is where you audit each of them

Align, correct & dedup, call cells, report — web_summary audits all four

Cell calling starts from the knee plot

barcode rank plot



Step 1: keep high-UMI barcodes

a hard cutoff from the expected cell count keeps the plateau

Step 2: rescue low-UMI cells

estimate the ambient profile from the shelf; low-UMI barcodes that don't look ambient are rescued (EmptyDrops-style test)

**cells with little RNA (e.g. T cells)
are not discarded just for low UMIs**

simulated data

An EmptyDrops-style two-step: keep the plateau, then rescue barcodes that don't look ambient (Lun et al. 2019)

CORE IDEA

**The computer runs the command.
Reading the report is your job.**

count almost never fails loudly — it happily produces a finished-but-unusable result. Reading the report is the only defence.

02

Anatomy of the command

Eight lines — and a reason for every one

One complete count run

```
cellranger count \  
  --id=pbm_c1k_count \  
  --transcriptome=refdata-gex-GRCh38-2020-A \  
  --fastqs=fastq/pbm_c1k_v3_fastqs \  
  --sample=pbm_c1k_v3 \  
  --create-bam=true \  
  --localcores=8 \  
  --localmem=64
```

That's all eight lines. Now we take them apart one by one

What each flag does

**cellranger count **

`--id=pbmc1k_count`

→ output folder name you choose; rerun needs a new one

`--transcriptome=refdata-gex-GRCh38-2020-A`

→ the reference folder from B2 (genome + GTF)

`--fastqs=fastq/pbmc_1k_v3_fastqs`

→ the folder holding the FASTQs, not one file

`--sample=pbmc_1k_v3`

→ the sample prefix in FASTQ names — must match exactly

`--create-bam=true`

→ required since v8: whether to write the BAM (large)

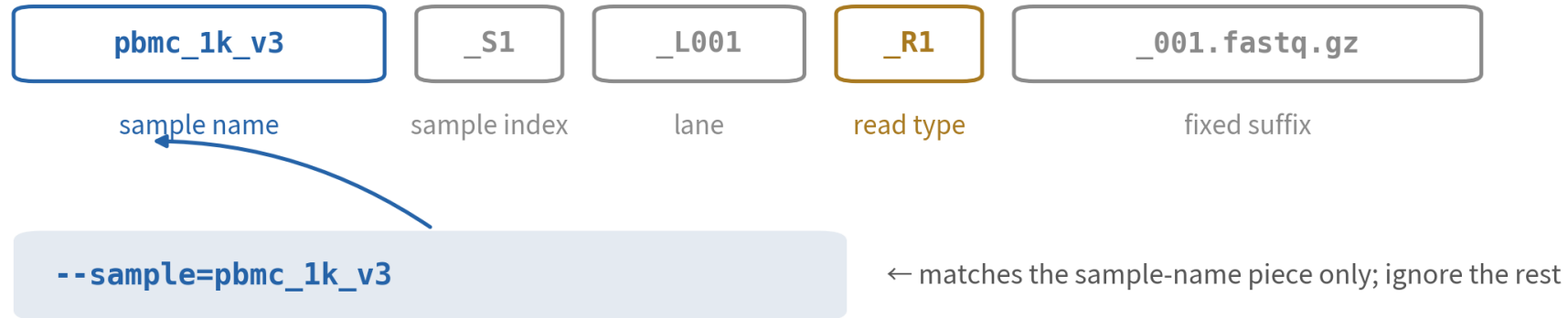
`--localcores=8 --localmem=64`

→ cores and GB of RAM to use; default grabs the whole machine

Blue: copy as-is. Gold: the two most error-prone. Red: affects the whole machine

--sample matches the FASTQ filename

the fixed anatomy of a FASTQ filename



a mismatch (e.g. `--sample=pbmc1k`) → "No input FASTQs were found" — it stops on the start line

- FASTQ names are fixed: `sample_S1_L001_R1_001.fastq.gz`
- `--sample` matches the sample-name piece only — exactly
- with several samples in one folder, it is the file selector

What a mismatch looks like — for real

```
# [踩雷示範] 樣本名打成 pbm c1k (漏了底線與版本)
cellranger count --id= trap_run \
  --transcriptome= refdata-gex-GRCh38-2020-A \
  --fastqs= fastq/pbm c_1k_v3_fastqs \
  --sample= pbm c1k --create-bam = true
```

error: No input FASTQs were found for the
requested parameters.

(檢查三件事: --fastqs 路徑、--sample 拼字、檔名前綴)

Good news: this error appears within 30 seconds, not after two wasted hours

Resources, and when --expect-cells matters

```
# 資源：不設限，它就吃滿整台機器——共用伺服器上是公害
--localcores=8      # 給幾顆核心
--localmem=64        # 給幾 GB 記憶體；人類參考建議 >= 64

# 預期細胞數：預設自動估，多數情況「不要給」
--expect-cells=5000 # 只在 cellcalling 明顯判錯、
                   # knee plot 又亂時，用上機數救回
```

--expect-cells is a correction tool, not a required field — run defaults first, then read the report

4-1 recap: three checks before Enter

Two paths



--transcriptome to the
B2 reference; --fastqs
to the folder

Prefix matches



--sample equals the
FASTQ prefix exactly

Resources capped



set localcores /
localmem explicitly

03

After you press Enter

Know what's scrolling by, and the wait stops being scary

What a running count looks like

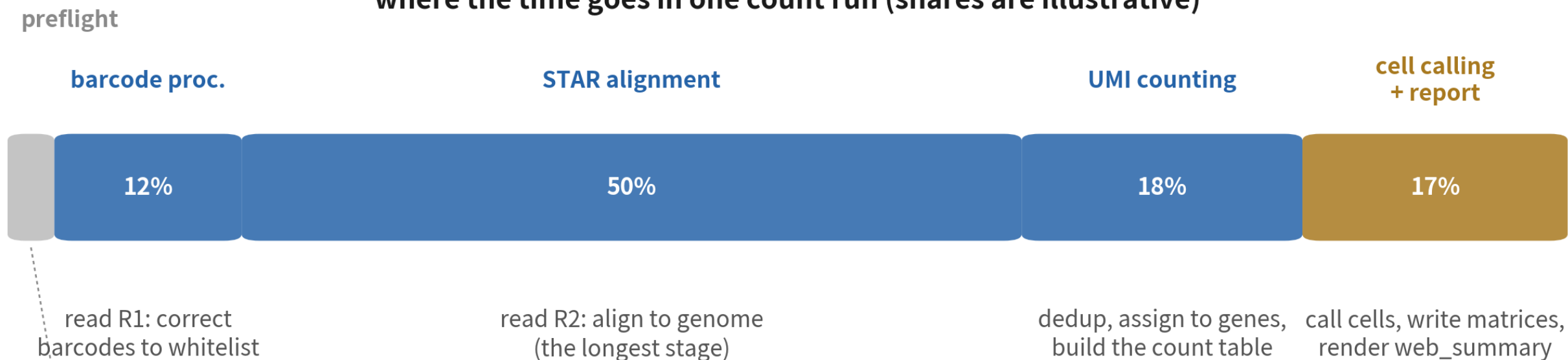
```
$ cellranger count --id=pbm c1k_count ...  
Martian Runtime - v4.0.13  
Serving UI at http://localhost:3600 ...  
Running preflight checks (please wait)...  
2026-08-18 09:12:03 [runtime] (run:local) ID .pbm c1k_count...  
# (接下來是幾百行 stage 訊息，一個 stage 一段，放著跑就好)
```

```
Waiting 6 seconds for UI to do final refresh.  
Pipestance completed successfully!
```

The line you want is the last one — no 'completed successfully', no outs/

Where the time goes

where the time goes in one count run (shares are illustrative)



checks args,
ref, FASTQs

ballpark: PBMC 1k takes ~1–2 h on 8 cores / 64 GB; below 32 GB STAR cannot even load the reference

STAR alignment dominates; if your machine can't run it, download the official finished outputs (Appendix C)

4-2 recap: what to do during the wait

Don't stare



mid-run failures are rare; errors show in the first 5 min

Write the log



note cells loaded and sample condition — you'll need them

Don't kill it



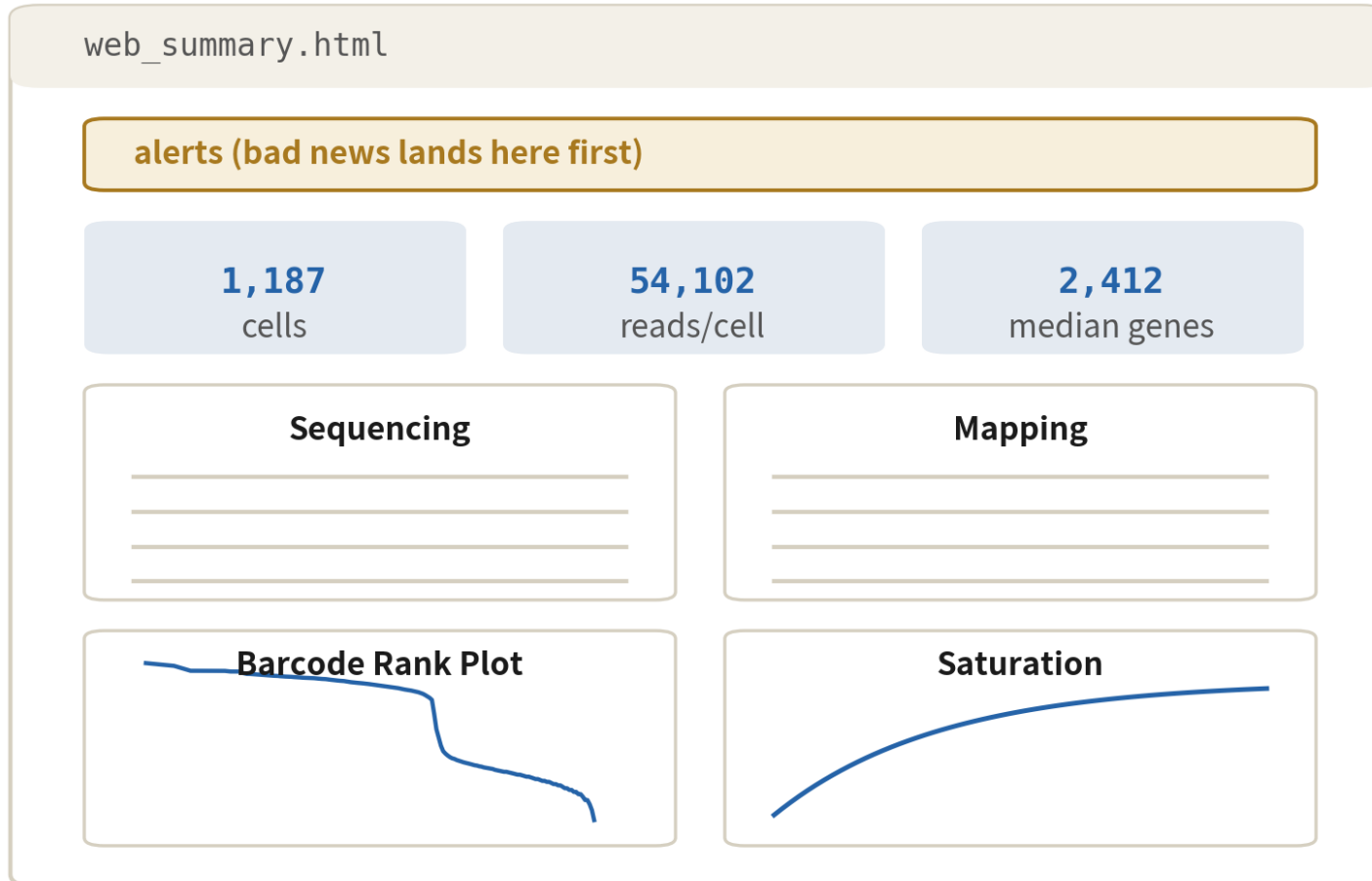
interrupt with Ctrl-C so it cleans up properly

04

Reading the web_summary, zone by zone

The heart of this episode: one page, one fixed reading order

The map: four zones, one fixed order



1. alerts first

a red ERROR makes the run a suspect

2. then the three hero numbers

does the cell count match what you loaded?

3. read the knee plot shape

is the cliff sharp? is the shelf clean?

4. mapping & saturation

where reads went; is depth sufficient?

schematic

Alerts → hero numbers → knee plot → mapping & saturation. Every report, same order

The three hero numbers

Estimated Number of Cells

1,187

cells identified by cell calling

healthy: near the expected recovery
(1,600 loaded → expect ~1,000)
chase it if far off in either direction

Mean Reads per Cell

54,102

total reads / cell count

this measures sequencing spend, not quality;
10x recommends at least 20,000 per cell
inflated cell counts dilute it

Median Genes per Cell

2,412

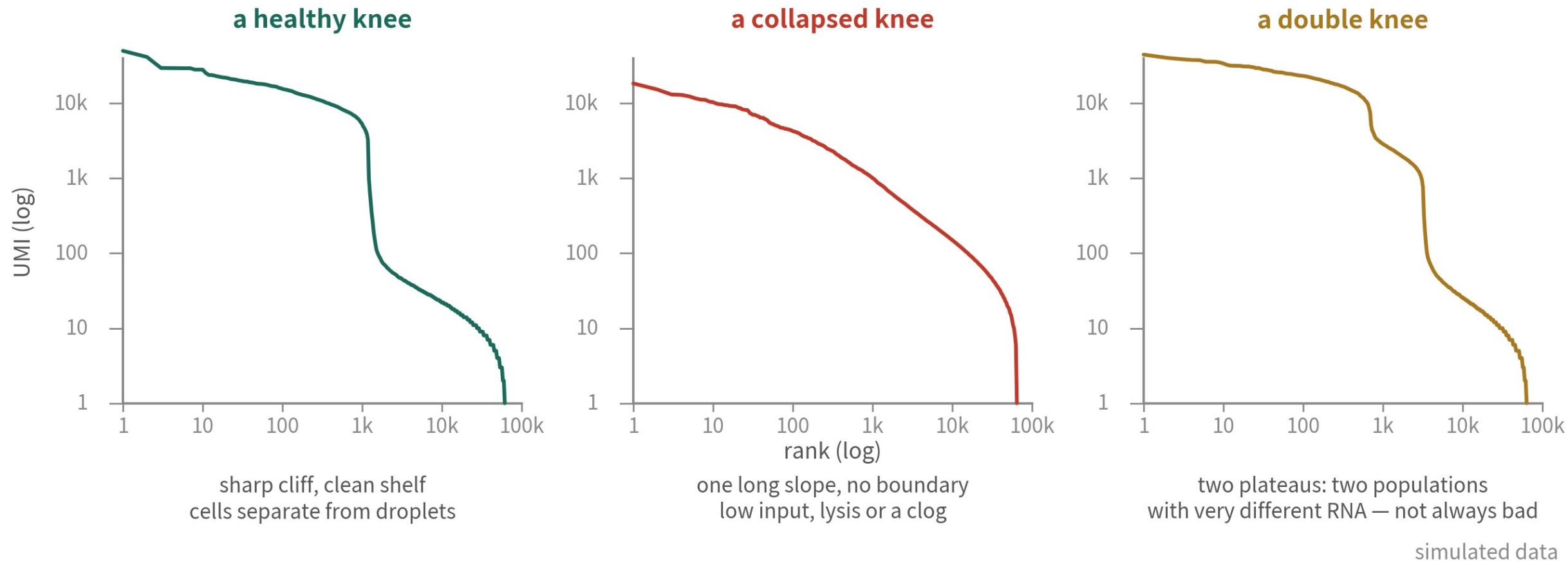
median genes detected per cell

depends on tissue, depth, chemistry;
PBMC at standard depth: ~1,000–2,500
a crash means low depth or inflated cells

the three move together: inflate the cell count and the other two dilute — never read one alone

- Estimated Cells: compare with loading; chase big gaps either way
- Mean Reads per Cell: a depth metric; 10x suggests $\geq 20,000$
- Median Genes per Cell: tissue- and depth-dependent; a crash is a flag
- they move together: inflated cells dilute the other two

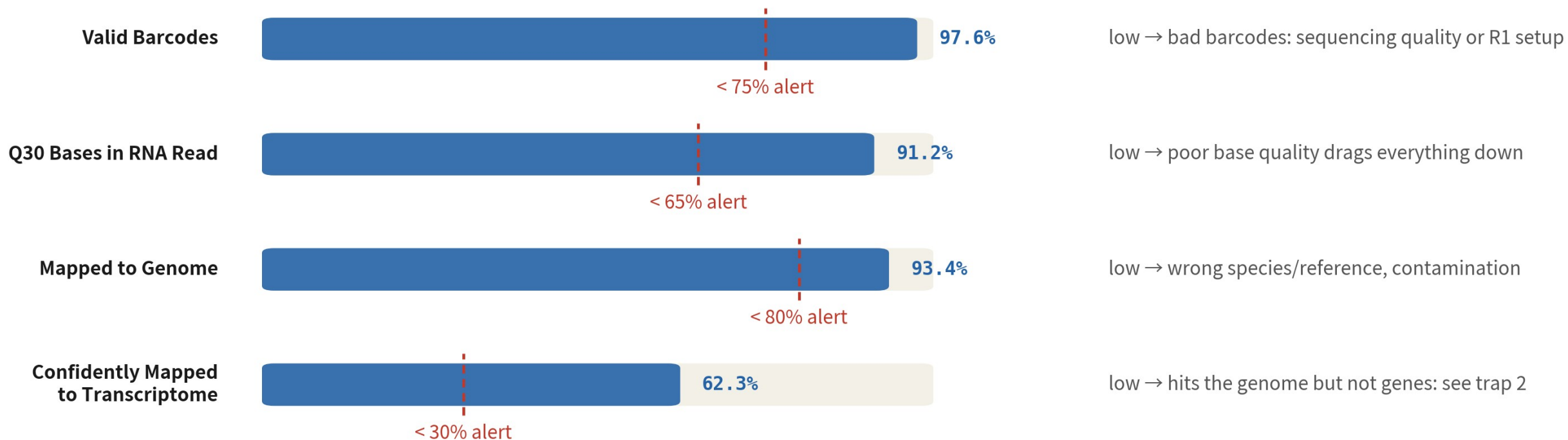
Knee-plot morphology



Healthy knee, collapsed knee, double knee — the shape tells the truth before the numbers do

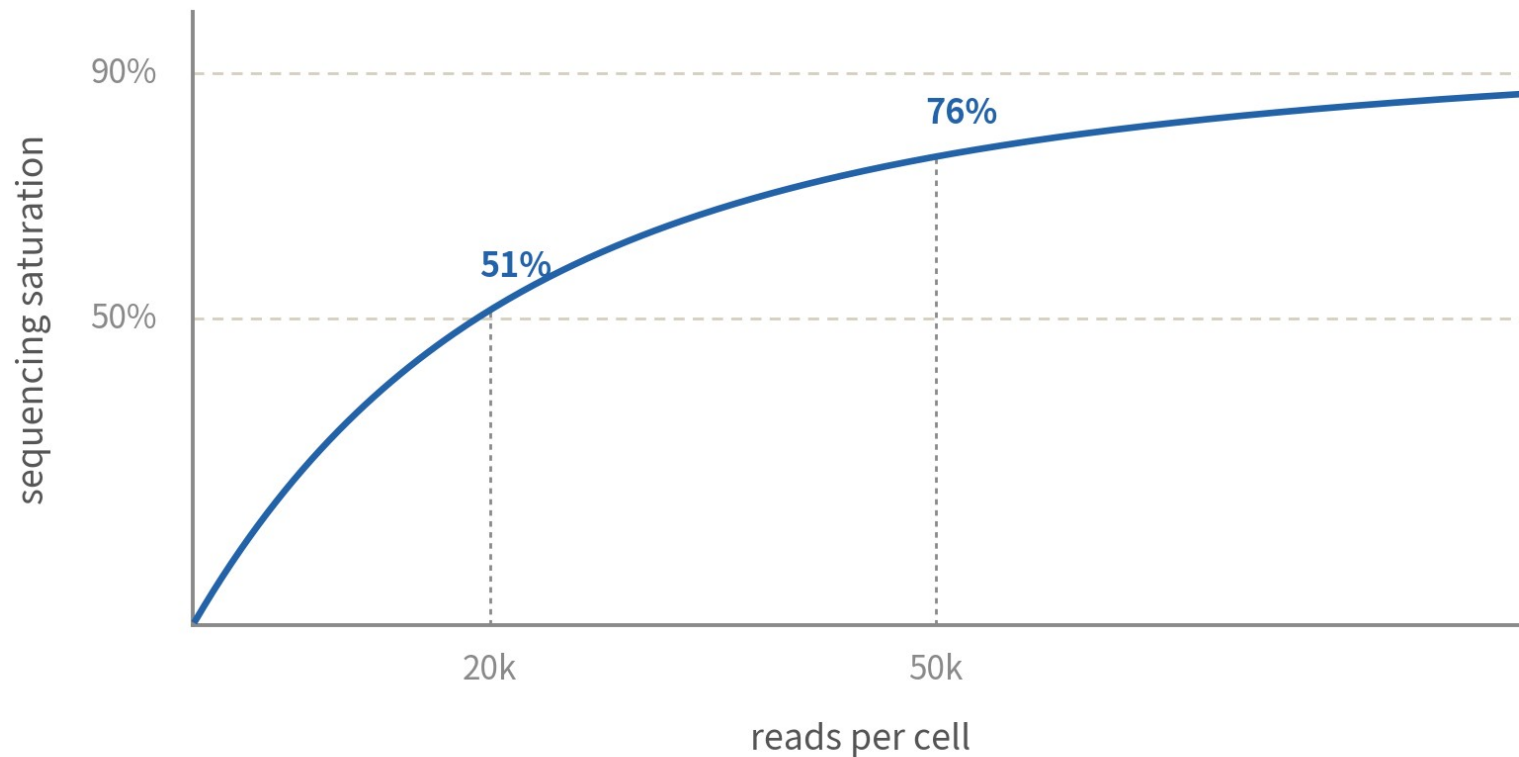
The mapping block and Q30

values are PBMC-style examples; red dashes mark common reference cutoffs (10x alerts sit nearby)



Note: include-introns defaults to on since v7, counting intronic reads — mind cross-version comparisons

What sequencing saturation means



what saturation means

"the chance the next read is a duplicate molecule"
50% = every second read is an already-seen UMI

there is no pass mark

high = library mined deeply
low = molecules still unseen
low is not failure — it means more depth would still pay

simulated data

It measures how deeply the library was mined — it is not a pass/fail exam

Low saturation: top up or not?

Ask what your biology needs before spending

hold

median genes already suffice

- markers, clustering, annotation: enough
- save budget for replicates instead
- Appendix F has the arithmetic

top up

the signal is genuinely short

- you need low-expression genes or rare transcripts
- low saturation → each new read still pays
- the same library can simply be resequenced

FOR YOUR ANALYSIS NOTES

**"How many alerts; does the cell count add up;
is the knee shape right; where did the reads map."**

Only when you can answer all four have you actually read the report.

05

Dissecting the outs/ folder

A dozen files — know who each one is for

List it first

```
$ ls outs/
```

```
analysis/                possorted_genome_bam.bam  
cLoupe.cLoupe            possorted_genome_bam.bai  
filtered_feature_bc_matrix/ raw_feature_bc_matrix/  
filtered_feature_bc_matrix.h5 raw_feature_bc_matrix.h5  
metrics_summary.csv      web_summary.html  
molecule_info.h5
```

It looks like a lot, but there are only four kinds: reports, matrices, molecule records, alignments

Who each file is for

outs/

web_summary.html

the health report — open it first

metrics_summary.csv

the same metrics as csv, for pipelines

filtered_feature_bc_matrix/

cells only — this is what Seurat reads

raw_feature_bc_matrix/

every whitelist barcode; nearly all empty

molecule_info.h5

one record per molecule; aggr needs it

possorted_genome_bam.bam

alignments (GBs); for velocity / re-analysis

cLoupe.cLoupe

for browsing in Loupe Browser

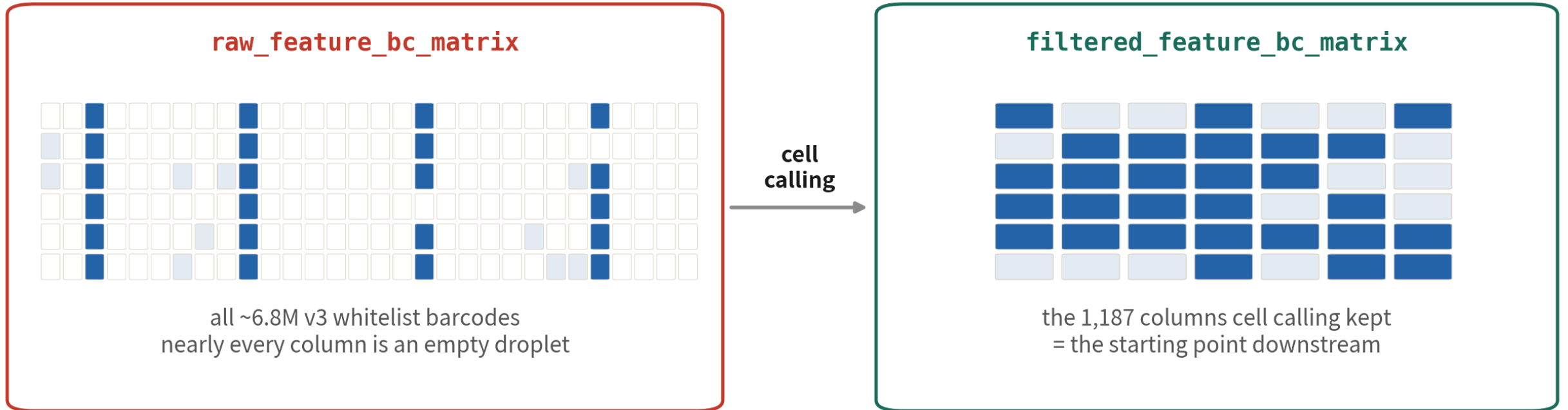
analysis/

Cell Ranger's own clustering/DR (reference only)

both matrix folders hold the same trio: barcodes.tsv.gz + features.tsv.gz + matrix.mtx.gz (also shipped as a single .h5)

B5 wants the filtered matrix; B4's aggr wants molecule_info.h5; the BAM serves advanced analyses

filtered vs raw: never confuse them



feed raw into Seurat and you get millions of "cells" — demonstrated in a moment

Two views of the same run: raw holds every barcode; filtered holds the survivors of cell calling

Grab the wrong one? Watch what happens

```
# [踩雷示範] 在 R 裡把 raw 當 filtered 讀  
library(Seurat)  
raw <- Read10X("outs/raw_feature_bc_matrix/")  
dim(raw)  
filt <- Read10X("outs/filtered_feature_bc_matrix/")  
dim(filt)
```

```
[1] 36601 6794880 # raw : 680 萬欄，幾乎全是空滴  
[1] 36601 1187 # filtered : cell calling 留下的細胞
```

6.79 million vs 1,187 — running out of memory is the lucky outcome; the scary one is when it doesn't

How to read the alerts

Two severities, two responses

WARN

Yellow: proceed with suspicion

- e.g. lowish saturation or gene counts
- usually usable — but write it down
- revisit when interpreting downstream

ERROR

Red: stop first

- e.g. low valid barcodes, collapsed mapping
- find the cause: sample, sequencing, reference?
- do not go downstream before you understand it

4-4 recap: three files leave with you

filtered matrix



what B5's Read10X will
eat — the downstream
start

web_summary



archive it; methods
sections cite these
numbers

molecule_info.h



5

multi-sample aggr in
B4 requires it

06

Where people get hurt

Three traps — each with its consequences laid bare

Trap 1: more cells than you loaded

What it is

You loaded 5,000 cells, web_summary says 9,315, and you take it at face value.



Why it happens

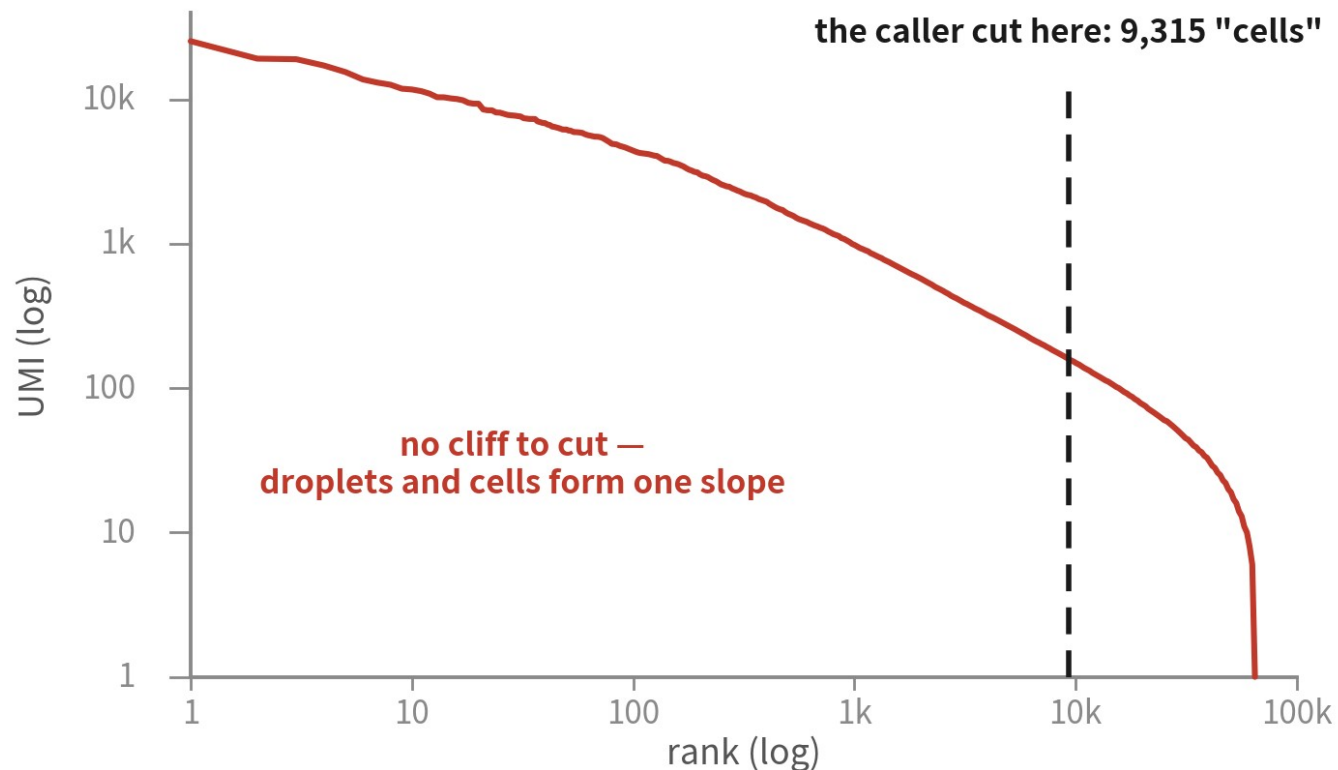
"The software computed it" sounds more trustworthy than your own lab notes.

What it costs you

Thousands of empty droplets enter the matrix as cells; QC saves only some, and clustering grows junk islands.

When recovery doesn't add up, read the knee plot

5,000 cells loaded, yet web_summary reports 9,315



why it happens

lysis, heavy ambient RNA, a clog,
or a wrong --expect-cells makes the
caller sweep the shelf into the cells

the one-line check

cells / loaded should land near 0.4–0.7;
if far off, read the knee plot first

the downstream cost

thousands of droplets pollute the matrix;
reads and genes per cell all dilute

simulated data

One-line check: cells / loaded should be ~0.4–0.7. Outside that, chase the cause before moving on

Trap 2: reading cells but never mapping

What it is

The cell count looks fine, so you declare success without ever scrolling to Mapping.



Why it happens

Hero numbers are big and on top; mapping is a table of small print.

What it costs you

Signals of a damaged sample — intronic spikes, antisense anomalies — are missed until DE, far too late.

The mapping breakdown tells the truth

even with a pretty cell count, this breakdown exposes a damaged sample (illustrative shares)

healthy whole-cell sample



a sample in trouble (lysis / degradation)



● Exonic ● Intronic ● Intergenic ● Antisense

cytoplasmic mRNA leaks away → nuclear unspliced RNA dominates → intronic spikes; the same shape can simply mean snRNA-seq

An exonic/intronic flip means cytoplasmic RNA was lost. The same shape is normal for snRNA-seq

Trap 3: panicking over low saturation

What it is

You see 33% saturation and immediately order double sequencing — or declare the run unusable.



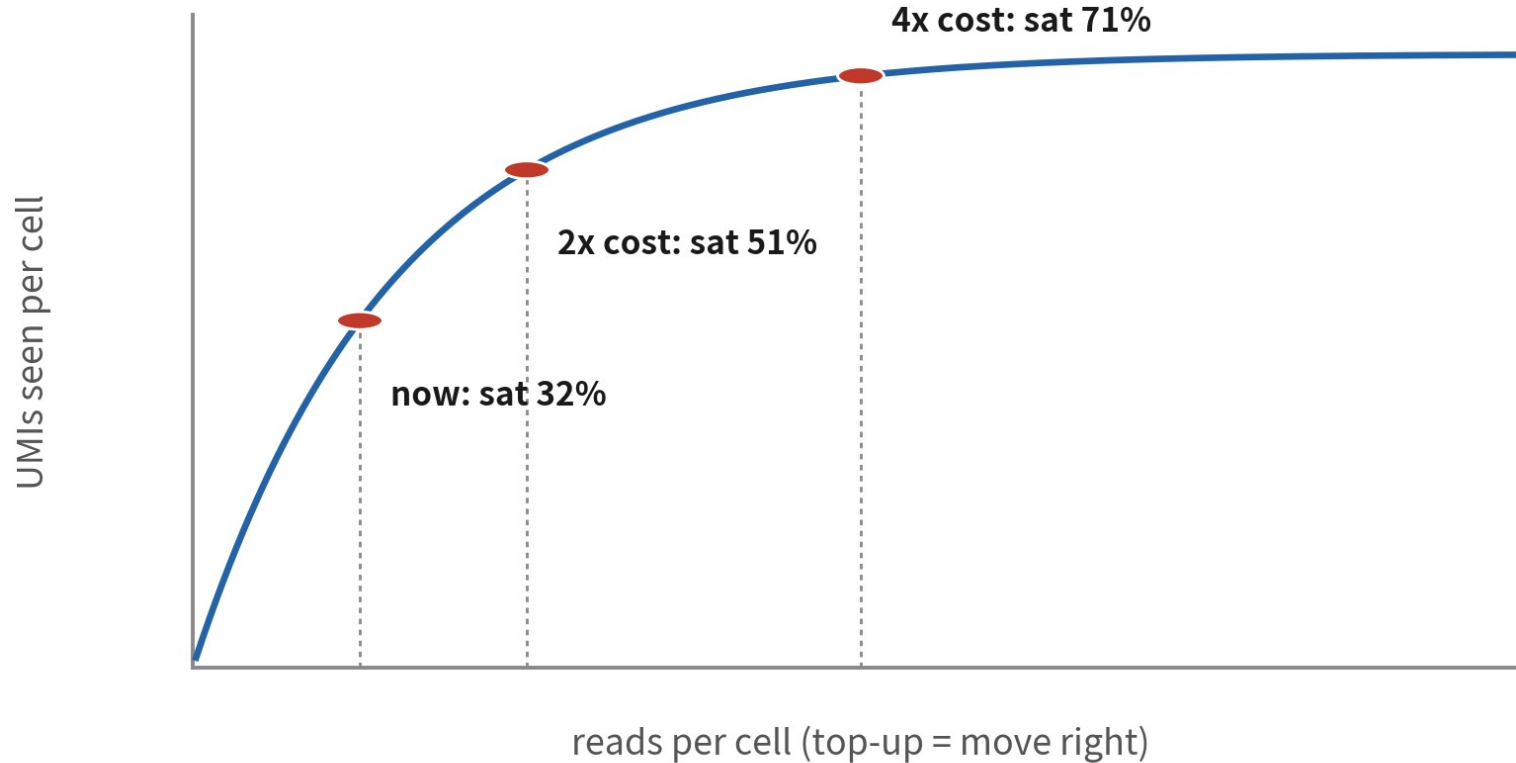
Why it happens

A percentage looks like an exam score, and 33 feels like a fail.

What it costs you

You pay double for a fifth more signal — or throw away data that was perfectly adequate. Both are waste.

The marginal return on topping up



do the arithmetic

double reads → +43% UMIs
double again → only +19%
(new genes grow even slower:
new molecules hit seen genes)

ask the right question

not "what's my saturation" but
"are median genes enough for
my biological question" —
if yes, don't top up

simulated data

Cost grows linearly, information in logs. The criterion is whether median genes answer your question — not the percentage

TRAPS, SUMMED UP

**Every web_summary number needs a reference point:
your loading notes, your tissue, your question.**

A number without a reference point is not interpretation — it is vibes.

05+

Strategy Room: Complex Samples

Same report, tumour sample — the whole set of reference points has to change

The tumour web_summary: normal abnormalities

Tumour sample • necrotic-core region • web_summary	
Estimated Number of Cells 8,000 loaded → 36% recovery	2,845
Median Genes per Cell far below the PBMC baseline	812
Fraction Reads in Cells < 70% raises an alert: heavy ambient	62.4%
Valid Barcodes technically healthy	97.1%
Mapped to Genome technically healthy	91.3%
alerts: 2 normal abnormality (expected) technically healthy	

schematic (illustrative numbers)

Low recovery and gene counts are what necrotic-core tissue looks like; technical metrics that crash still stop the run

"Normal abnormalities": swap the reference point

necrotic core = few live cells, thick soup:
low recovery, low genes, depressed Fraction
Reads in Cells are expected for this site, not
pipeline failures. The reference point is your own
history of the same tissue and site — not PBMC

The floor stands: technical crashes still stop you

Valid Barcodes, Q30, Mapped to Genome don't
care what the tissue looks like — when they crash,
blame sequencing or the reference, not the tumour

**an expectation is an assumption: how much, why,
based on which runs — write it down**

snRNA-seq: high intronic is a feature, not a warning

Frozen tumours usually run single-nucleus — same number, ask the design first

whole-cell scRNA

intronic around 20–30% is normal

- a surge = cytoplasmic mRNA lost (trap 2)
- include-introns mainly shifts the gene counts
- reference point: whole-cell baselines

single-nucleus snRNA

intronic at 40–50% is the signature

- frozen tissue: membranes broken — nuclei captured on purpose
- include-introns must stay on: off discards about half the signal
- on or off, and why — into the record

Different tumours, different recovery baselines

The 0.4–0.7 check line is a PBMC baseline — new tissue, new number

Sample	Tissue	What you see	How to adjust
PBMC (baseline)	suspension cells; no dissociation	recovery 0.4–0.7, clean knee	use the standard baseline as-is
Breast (fat-rich)	big fragile adipocytes burst in dissociation	lower recovery, heavier ambient	lower expected recovery; check past runs first
Liver (fibrotic)	dense matrix; harsher, longer dissociation	low recovery; composition skews to immune cells	note the composition bias downstream
GBM (necrotic core)	few live cells; thick ambient soup	low recovery, depressed Fraction Reads in Cells	lower and record the expectations (previous slides)

ONE-LINE SUMMARY

**count produces a report, not an answer;
whether the data proceeds is your call.**

Alerts, cell count, knee shape, mapping — clear all four gates before going downstream.

Check yourself 1

What separates the filtered matrix from the raw one?

- A filtered has gene-level filtering applied
- B filtered keeps only barcodes that cell calling deemed cells
- C filtered is already normalized
- D they are identical, just compressed differently

Answer B. The gene rows are identical; the columns differ: raw holds every whitelist barcode (~6.8M for v3), filtered holds the survivors of cell calling. Nothing is normalized — that's B6's job.

Check yourself 2

A barcode rank plot that is one continuous slope with no cliff most likely means...

- A the run is perfectly normal
- B cells and droplets are inseparable: low input, lysis, or a clog
- C sequencing was too deep
- D the wrong reference genome was used

Answer B. The cliff exists because cells carry far more RNA than droplets; a collapsed slope means the two blur together — lysis, low input, or a clog. Any cell-calling cut on such a plot is a guess.

Check yourself 3

Your web_summary: saturation 35%, median genes 2,100, and your goal is clustering plus annotating major populations. Top up?

- A Yes — 35% saturation is a failing grade
- B Not yet: 2,100 median genes support clustering; note it and proceed
- C Unusable — redo the whole experiment
- D Immediately resequence at triple depth

Answer B. Saturation is not an exam score. Clustering and major-type annotation work fine at ~2,000 median genes; 35% merely says more molecules remain. Record it, and only top up if your question escalates to low-expression genes — the same library can simply be resequenced.

NEXT EPISODE

B4: Multi-sample and multimodal

You can run one sample. What about four — four counts plus aggr, or one multi run? What does the wrong choice cost?

(Eager to reach Seurat? Skip ahead to B5 — Cell Ranger's "cells" still have to survive Seurat's scrutiny.)